

SIRA: Sistema Integral de Riego Automático

Dossier Técnico Maestro — Biblia del Proyecto (Versión 1.0 Final)

Proyecto	Trabajo de Fin de Grado — ASIR
Especialidad	Administración de Sistemas Informáticos en Red
Centro	IES / Centro de Formación Profesional
Fecha	Mayo 2026
Versión	1.0 Final

Equipo de Desarrollo:

Integrante	Rol Principal
Juan Risueño	Arquitectura, Backend (FastAPI), Seguridad (Iron Fortress), Cloud (AWS)
Jorge Pedro López	Coordinación, Diseño E/R, Frontend (PHP), Hardware/IoT
Alfonso Navarro	Base de Datos (SQL), Normalización, CSS, Documentación Técnica

Índice de Contenidos

1. Introducción y Marco Conceptual
2. Análisis de Requisitos y Caso de Uso Real
3. Modelado de Datos y Base de Datos
4. Arquitectura de Sistemas e Infraestructura
5. Arquitectura de Software: Backend (FastAPI)
6. Arquitectura de Software: Frontend (PHP)
7. Lógica de Dominio: Sensores, Actuadores y Cultivos
8. Protocolo de Seguridad "Iron Fortress"
9. Despliegue en Producción (AWS)
10. Análisis de Riesgos y Mitigaciones
11. Planificación y Línea Temporal
12. Auditoría Final y Conclusiones
13. Autoría y Roles del Proyecto

1. Introducción y Marco Conceptual

1.1 Visión del Proyecto

SIRA (Sistema Integral de Riego Automático) es una plataforma tecnológica diseñada para la optimización del riego y la monitorización climática en explotaciones agrícolas bajo invernadero. El sistema permite la gestión centralizada de múltiples fincas, parcelas e invernaderos desde un único panel de control web, automatizando la toma de decisiones a partir de los datos captados por sensores IoT en tiempo real.

El proyecto se enmarca en el contexto de la agricultura de precisión en las provincias de Almería y Murcia, donde los cultivos en invernadero representan una actividad económica de primer orden que exige un uso eficiente del agua y un control climático constante.

1.2 Filosofía de Desarrollo

El proyecto se rige por dos principios arquitectónicos fundamentales:

- **Zero-JS Policy:** El uso de JavaScript se limita al mínimo estrictamente necesario. Toda la lógica de negocio, validación crítica y seguridad reside en el servidor (Python/PHP). Esta decisión elimina una superficie de ataque significativa y garantiza que el sistema funcione en cualquier navegador.
- **Server-Side Rendering (SSR):** El frontend es generado íntegramente en el servidor por PHP. El navegador recibe HTML/CSS listo para mostrar, sin dependencias de frameworks de JavaScript.
- **Soberanía del Servidor:** Separación estricta de responsabilidades entre la capa de presentación (Frontend PHP), el núcleo de servicios (API FastAPI) y la persistencia de datos (PostgreSQL).

1.3 Contexto Tecnológico

El sistema se ha diseñado y probado en el contexto de la defensa de un TFG de ASIR. Dado que no se dispone de hardware físico (sensores reales), se ha implementado un simulador software (`simulador.py`) que inyecta datos sintéticos con variaciones realistas en la base de datos, permitiendo demostrar toda la lógica de automatización del sistema.

2. Análisis de Requisitos y Caso de Uso Real

2.1 Perfil del Cliente Objetivo

SIRA ha sido diseñado teniendo en mente al agricultor profesional con explotaciones de mediano tamaño. El caso de uso de referencia utilizado durante el desarrollo es el de **“Invernaderos El Sol de Almería S.L.”**, empresa ficticia gestionada por Antonio, propietario de dos fincas:

Finca	Ubicación	Invernaderos	Cultivos
“La Finca Grande”	Níjar, Almería (CP 04100)	Nave 1 (100x50m), Nave 2 (100x50m), Nave 3 (120x50m), Nave 4 (80x40m)	Tomate Pera (N1, N2), Pimiento California (N3), Calabacín (N4)
“Los Pinos”	Águilas, Murcia (CP 30880)	Nave A (90x40m), Nave B (90x40m)	Tomate Cherry (A y B)

2.2 Requisitos Funcionales Clave

- Registro y gestión multi-tenant de clientes, parcelas e invernaderos.
- Monitorización en tiempo real de sensores climáticos (temperatura, humedad, lluvia, viento, radiación solar).
- Automatización de actuadores (riego, ventanas, iluminación LED, extractores, calefacción) en función de umbrales por cultivo.
- Panel de control web con representación gráfica de datos de sensores (SSR con SVG).
- Sistema de roles: Root, Administrador, Cliente.
- Auditoría de seguridad: gestión de sesiones, historial de contraseñas, caducidad de credenciales.

2.3 Requisitos No Funcionales

- **Portabilidad:** El sistema debe poder desplegarse en local y en la nube (AWS) sin modificar el código fuente.
- **Seguridad:** Ninguna contraseña se almacenará en texto plano. Toda comunicación interna se validará mediante JWT.
- **Eficiencia:** El sistema debe funcionar en hardware modesto (instancia EC2 t2.micro con 1 GB de RAM).
- **Independencia:** Sin dependencias de APIs externas. La base de conocimientos agronómicos es propia y local.

3. Modelado de Datos y Base de Datos

3.1 Tecnología y Justificación

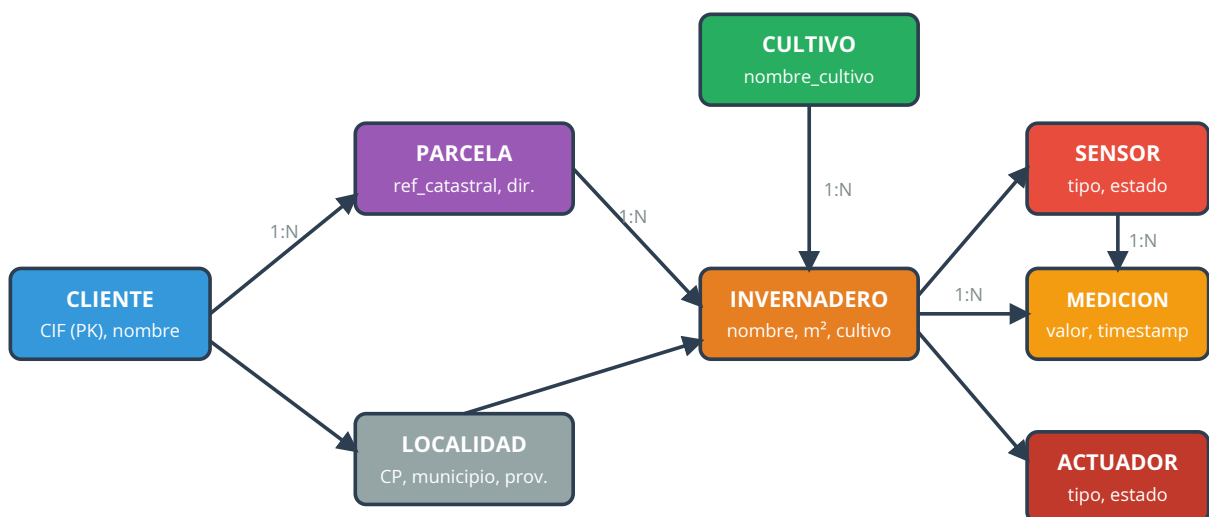
Se utiliza **PostgreSQL** como motor de base de datos relacional. La elección se justifica por su robustez, soporte nativo de tipos de datos avanzados (DECIMAL, TIMESTAMPTZ), y su excelente integración con SQLAlchemy (ORM del backend Python). Los datos de los sensores incluyen marcas de tiempo con zona horaria explícita (`TIMESTAMPTZ`) para evitar desajustes entre el horario UTC de los contenedores y la zona horaria local (Europe/Madrid).

3.2 Principios de Normalización

El esquema está normalizado hasta la **3FN (Tercera Forma Normal)**:

- No existen dependencias transitivas entre atributos no clave.
- Todas las relaciones son **1:N** (Uno a Muchos). Se ha evitado deliberadamente el uso de tablas pivote N:M para simplificar las consultas SQL en el contexto del TFG.
- El **borrado lógico** (campo `activa BOOLEAN`) se aplica en todas las entidades principales, preservando el historial de datos sin eliminar registros físicamente.

3.3 Diagrama Entidad-Relación (Simplificado)



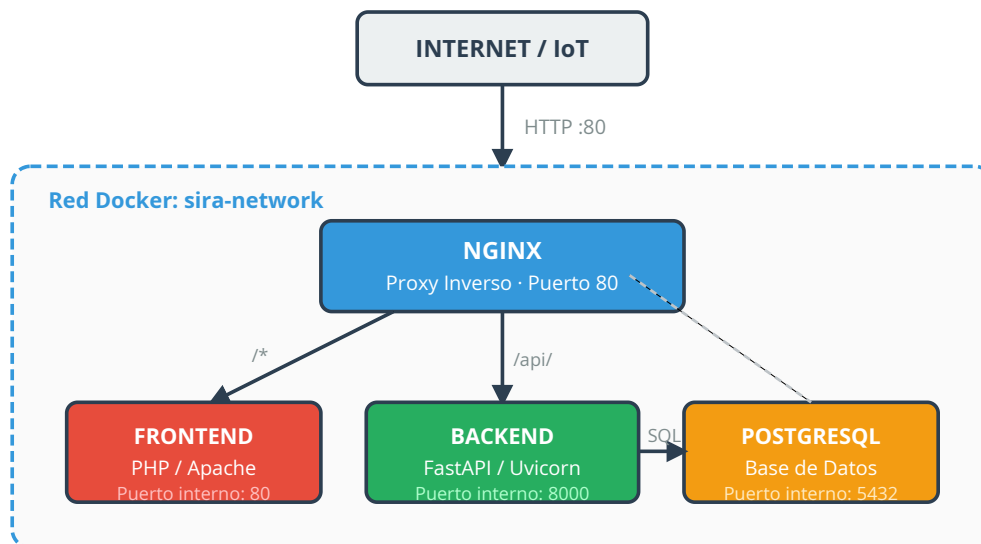
3.4 Entidades Principales

Entidad	Clave Primaria	Descripción
CLIENTE	cliente_id	Empresa/autónomo propietario de las fincas. Accede con su CIF.
LOCALIDAD	codigo_postal	Catálogo de municipios. Normaliza la ubicación de las parcelas.
PARCELA	parcela_id	Finca física identificada por referencia catastral.
INVERNADERO	invernadero_id	Nave dentro de una parcela. Contiene sensores y actuadores.
CULTIVO	cultivo_id	Tipo de cultivo con sus parámetros óptimos asociados.
PARAMETROS_OPTIMOS	parametro_id	Rangos ideales (temp., humedad, pH) por cultivo y fase.
SENSOR	sensor_id	Dispositivo de lectura (temperatura, humedad, viento, etc.).
MEDICION	medicion_id	Registro histórico de una lectura de sensor con timestamp.
ACTUADOR	actuador_id	Dispositivo de control (riego, ventana, calefacción, etc.).
ACCION_ACTUADOR	accion_id	Log de cada acción ejecutada por un actuador.
RECOMENDACION_RIEGO	recomendacion_id	Decisión de riego generada automáticamente por el backend.

4. Arquitectura de Sistemas e Infraestructura

4.1 Visión General

SIRA se despliega como un conjunto de microservicios orquestados mediante **Docker Compose**. Todos los componentes operan dentro de una red interna privada (`sira-network`), siendo Nginx el único servicio expuesto al exterior.



4.2 Nginx como API Gateway y Perímetro de Seguridad

Nginx es la pieza central de la infraestructura de red. Su configuración implementa:

- **Aislamiento de puertos:** Los puertos internos de FastAPI (8000), PostgreSQL (5432) y Apache (80 interno) nunca se exponen a internet.
- **Enrutamiento inteligente:** Las peticiones a `/api/`, `/docs` y `/redoc` se redirigen al contenedor FastAPI. El resto del tráfico se sirve desde el contenedor PHP.
- **Prevención DoS:** La directiva `client_max_body_size 50M` previene ataques de saturación por payload.
- **Soporte WebSocket:** Habilitado mediante las cabeceras `Upgrade` para el flujo continuo de datos IoT.
- **Abstracción de entorno:** El puerto de exposición se configura mediante variable de entorno (`SIRA_PORT`), permitiendo usar 8085 en local y 80 en AWS sin tocar el código.

4.3 Variables de Entorno (.env)

Toda la configuración sensible se centraliza en un archivo `.env` excluido del control de versiones (`.gitignore`):

Variable	Descripción	Valor Local	Valor AWS
SIRA_PORT	Puerto de exposición de Nginx	8085	80
DB_USER	Usuario de PostgreSQL	juanrisueno	(seguro)
ACCESS_TOKEN_EXPIRE_MINUTES	Duración máxima de sesión	1440 (24h)	1440
DB_PASSWORD	Contraseña de PostgreSQL	(local)	(seguro)

4.4 Estrategia de Copias de Seguridad (Backups)

Para garantizar la integridad de los datos ante fallos críticos o errores humanos, se ha implementado un sistema de **backups graduales inteligentes** mediante el script `scripts/backup_sira.sh`.

Lógica de Decisión (Cerebro de Backup)

El script no se limita a copiar archivos, sino que analiza el historial para optimizar el espacio en disco:

1. **Copia Anual:** Si es la primera ejecución del año, genera un backup en la carpeta `/anuales`. Se conservan las últimas **2 copias**.
2. **Copia Mensual:** Si ya hay anual pero es el primer backup del mes, se guarda en `/mensuales`. Se conservan las últimas **3 copias**.
3. **Copia Diaria:** En cualquier otro caso, genera un backup en `/diarios`. Se conservan las últimas **10 copias**.

Características Técnicas

- **Exclusiones Inteligentes:** Para optimizar el tamaño, el script ignora directorios pesados o innecesarios como `.git`, `venv`, `__pycache__` y los datos crudos de la base de datos (que se gestionan mediante volcados SQL independientes).
- **Prioridad de Sistema:** Utiliza el comando `nice -n 19` para realizar la compresión con la menor prioridad de CPU, evitando que el proceso de backup afecte al rendimiento de la API en producción.
- **Autogestión de Espacio:** Implementa una lógica de **rotación automática** que elimina las copias más antiguas una vez superado el límite configurado para cada tipo.
- **Metadatos:** Cada copia incluye un archivo `info_backup.txt` con la fecha, el usuario que lo lanzó y el tipo de backup realizado.

Automatización (Crontab)

Para asegurar la ejecución sin intervención humana, el script se programa en el `crontab` del sistema (Ubuntu en AWS). Se recomienda una ejecución diaria a una hora de baja carga (ej. 03:00 AM):

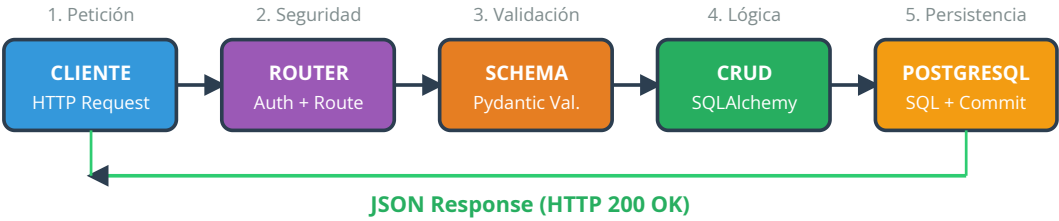
```
# Programación en crontab -e
00 03 * * * /bin/bash /home/ubuntu/SIRA_Project/scripts/backup_sira.sh >> /home/ubuntu/sira_b
```

5. Arquitectura de Software: Backend (FastAPI)

5.1 Stack Tecnológico del Backend

Componente	Tecnología	Rol
Framework Web	FastAPI (Python)	Exposición de endpoints REST y documentación automática
Servidor ASGI	Uvicorn	Motor de ejecución asíncrona
ORM	SQLAlchemy	Mapeo objeto-relacional hacia PostgreSQL
Validación	Pydantic (Schemas)	Validación de datos de entrada y serialización de salida
Seguridad	python-jose + passlib	Generación y verificación de JWT / hashing Bcrypt

5.2 Flujo de Datos Interno (Request/Response)



5.3 Organización de Archivos del Backend

Archivo	Responsabilidad
main.py	Punto de entrada. Registra routers y configura middleware.
models.py	Define las tablas de PostgreSQL como clases Python (SQLAlchemy).
schemas.py	Define los modelos de validación Pydantic para entrada y salida de datos.
crud.py	Implementa las operaciones de lectura, escritura, actualización y borrado.
routers/	Directorio con un archivo por recurso (usuarios, parcelas, sensores, etc.).
security.py	Lógica de generación y verificación de tokens JWT + Bcrypt.

5.4 Documentación Automática (Swagger)

FastAPI genera automáticamente documentación interactiva accesible en `/docs` (Swagger UI) y `/redoc`. Estos endpoints están protegidos por Nginx para que solo sean accesibles desde la red interna o mediante VPN en producción.

6. Arquitectura de Software: Frontend (PHP)

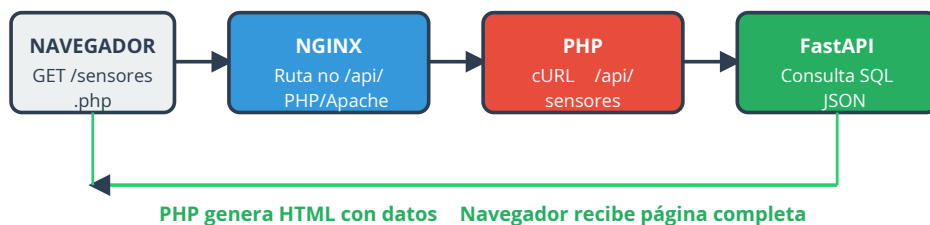
6.1 Rol del Frontend y Filosofía SSR

El frontend de SIRA está construido en **PHP con renderizado en servidor (SSR)**. Esta decisión arquitectónica implica que el servidor genera el HTML completo antes de enviarlo al navegador, garantizando que:

- La lógica de negocio (permisos, roles, filtros de datos) nunca se expone al cliente.
- El sistema funciona en cualquier navegador, incluso con JavaScript desactivado.
- La carga inicial es más rápida, ya que no hay que esperar a que un framework de JavaScript hidrate la página.

6.2 PHP como Proxy/Gateway hacia la API

La comunicación con el backend no se realiza desde el navegador, sino desde PHP en el servidor mediante la librería **cURL**. El flujo completo para servir una página de datos es:



6.3 Sistema de Diseño Visual (CSS)

Se ha implementado una arquitectura CSS modular y basada en **Custom Properties (Variables CSS)**:

- **variables.css** : Token de diseño central. Color primario SIRA: `#10b981` (verde esmeralda). Fondo: `#0f172a` (azul marino oscuro). Modo oscuro por defecto.
- **base.css** : Reset de estilos + tipografía (Google Fonts: Inter, Roboto).
- **layout.css** : Estructura de navegación y cuerpo principal.
- **Archivos espejo**: Cada página PHP tiene su propio `nombre.css` para estilos específicos.
- **Módulos de clima**: `rain.css`, `heat.css`, `snow.css`, `cloudy.css` — cargados dinámicamente según los datos del sensor. Los efectos usan `pointer-events: none` para no bloquear la interacción.

6.4 Control de Acceso en el Frontend

- **header.php** : Incluido en todas las páginas. Extrae el rol del usuario del JWT y muestra/oculta elementos de la interfaz en consecuencia (Root > Admin > Cliente).
- **Borrado lógico**: Se usa el campo `activa = false` en lugar de eliminar registros, preservando el historial.
- **Refresco automático**: La etiqueta `<meta refresh>` recarga las páginas de monitorización cada 10-15 segundos. Está desactivado en páginas con formularios para no perder datos del usuario.

7. Lógica de Dominio: Sensores, Actuadores y Cultivos

7.1 Dispositivos Implementados

Sensores (Entrada de datos):

Sensor	Variable medida	Unidad
Temperatura	Temperatura interior del invernadero	°C
Humedad del Suelo	Nivel de humedad del sustrato	%
Lluvia	Detección de precipitación	Booleano
Viento	Velocidad del viento exterior	km/h
Radiación Solar	Intensidad de luz recibida	W/m ²

Actuadores (Salida / Control):

Actuador	Acción	Trigger
Riego (Electroválvula)	Apertura/cierre del suministro de agua	Humedad suelo < 60%
Motor de Ventana	Apertura/cierre de ventilación	Temp > 30°C o viento > 45km/h
Iluminación LED	Encendido/apagado de luces	Radiación < 200 W/m ² en jornada laboral
Extractor de Aire	Renovación del aire	Humedad relativa > 90%
Calefacción	Protección contra heladas	Temperatura < 10°C

7.2 Lógica de Automatización y Prioridades

El backend evalúa las mediciones entrantes y aplica las siguientes reglas por orden de prioridad:



7.3 Parámetros Agronómicos por Cultivo

Los umbrales de automatización se adaptan al cultivo configurado en cada invernadero:

Cultivo	Temp. Óptima (°C)	Humedad Óptima (%)	pH Ideal	Riego (L/m²/día)
Tomate	18 – 30	60 – 80	6.0 – 6.8	5.0
Pimiento	21 – 26	65 – 85	5.5 – 7.0	4.5
Pepino	20 – 30	70 – 90	5.5 – 7.0	4.2
Sandía	20 – 30	60 – 70	6.0 – 6.8	3.8
Melón	20 – 30	60 – 70	6.0 – 6.8	3.8
Calabacín	25 – 35	65 – 80	5.6 – 6.8	4.0
Berenjena	25 – 30	60 – 80	5.5 – 6.8	5.0

Fuentes: Fundación Cajamar, MAPA, InfoAgro Almería.

7.4 Control Manual y “Modo Cortesía”

Si el usuario activa o desactiva un actuador manualmente desde el panel, el sistema automático **respetará esa decisión durante 2 horas**. Pasado ese tiempo, la automatización se reactiva para prevenir descuidos (ej. riego abierto toda la noche).

7.5 Estrategia de Simulación IoT

Justificación de la Decisión

El sistema SIRA está diseñado para integrarse con sensores físicos reales (temperatura, humedad, viento, lluvia, radiación solar) desplegados en invernaderos. Sin embargo, en el contexto de un Trabajo de Fin de Grado, **no se dispone del hardware físico necesario** (microcontroladores, módulos de comunicación, cableado de campo, etc.) para realizar una demostración con dispositivos reales ante el tribunal evaluador.

Como solución técnica a esta limitación, se ha desarrollado un **simulador por software** (`scripts/simulador.py`) que replica fielmente el comportamiento de una red de sensores IoT real:

- **Inserción periódica:** El script se conecta directamente a PostgreSQL e inserta nuevas mediciones cada 10 segundos, simulando el ritmo de telemetría de sensores físicos.
- **Realismo estadístico:** Los valores no son constantes, sino que incorporan pequeñas variaciones aleatorias (ruido gaussiano) para evitar que las gráficas del dashboard muestren líneas rectas artificiales.
- **Respuesta automática en tiempo real:** El backend de FastAPI analiza cada nueva medición y decide en el momento si debe activar o desactivar actuadores, exactamente igual que haría con hardware real.
- **Base de conocimientos local:** Se ha descartado el uso de APIs externas de datos agronómicos (como Perenual) para garantizar que el sistema funcione al 100% sin conexión a internet durante la defensa. Los parámetros óptimos por cultivo están almacenados en la base de datos propia.

Esta decisión demuestra que la **arquitectura del sistema es agnóstica a la fuente de datos**: en un despliegue productivo real, únicamente sería necesario sustituir el script simulador por los drivers de comunicación del hardware (MQTT, HTTP, Modbus), sin modificar ninguna línea del backend ni del frontend.

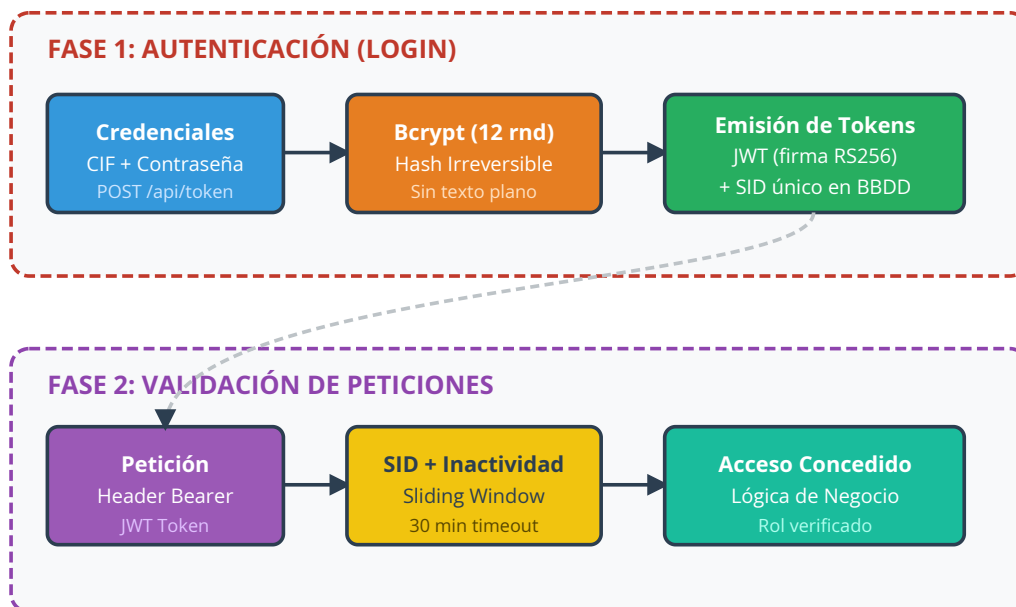
Escenarios de Demostración (Presets)

El simulador incluye los siguientes escenarios climáticos pre-configurados, activables durante la defensa para mostrar la respuesta automática del sistema:

Preset	Condición Simulada	Respuesta Esperada del Sistema
Condiciones Ideales	Temp 22°C, Hum 70%, sin viento	Actuadores en reposo
Tormenta	Viento > 60 km/h + lluvia	Cierre inmediato de ventanas (Prioridad 1)
Ola de Calor	Temp 42°C, Hum 30%	Extractores ON + riego ON + alerta usuario
Helada Nocturna	Temp 3°C	Calefacción ON (Prioridad 2)
Día Nublado	Radiación 50 W/m²	Luces LED ON (si es horario laboral)
Randomize	Valores aleatorios en rangos amplios	Evaluación dinámica de la lógica de control ante ambientes impredecibles

8. Protocolo de Seguridad “Iron Fortress”

La seguridad es un eje transversal en todo SIRA, implementada bajo el nombre en clave **Iron Fortress**.



8.1 Mecanismos de Seguridad Implementados

Mecanismo	Detalle
Bcrypt (12 rondas)	Hashing irreversible de contraseñas. Ningún script SQL contiene contraseñas en texto plano.
JWT Stateless	Tokens firmados con clave secreta. Contienen el rol del usuario en los claims. Duración máxima: 24h.
Session ID (SID) único	Cada login genera un SID que se almacena en BD. Un nuevo login invalida el SID anterior (previene sesiones duplicadas).
Sliding Window (30 min)	Si no hay actividad en 30 minutos, la sesión se cierra automáticamente.
Historial de contraseñas	Se guardan las últimas 5 contraseñas (en JSON fuera de la BD) para evitar su reutilización.
Caducidad de credenciales	Las contraseñas caducan a los 90 días.
XSS Prevention	PHP usa <code>htmlspecialchars()</code> en todos los datos mostrados al usuario.
SQL Injection Prevention	SQLAlchemy parametriza todas las consultas. No existe SQL concatenado en cadenas.
Almacenamiento seguro del JWT	El token se guarda en la sesión PHP del servidor, no en localStorage del navegador.
Secretos fuera del repositorio	El archivo <code>.env</code> está en <code>.gitignore</code> . Nunca se sube a GitHub.

8.2 Matriz de Riesgos

Riesgo	Impacto	Probabilidad	Medida
Inyección SQL	Crítico	Baja	ORM SQLAlchemy parametrizado
Robo de sesión (XSS)	Alto	Baja	JWT en sesión PHP + <code>htmlspecialchars()</code>
Fuerza bruta	Alto	Media	Bcrypt 12 rondas + contraseñas complejas
Acceso no autorizado	Crítico	Baja	Validación de rol en cada endpoint
Fuga de secretos	Crítico	Baja	<code>.gitignore</code> + variables de entorno
Sesión olvidada abierta	Medio	Alta	Sliding Window 30 min

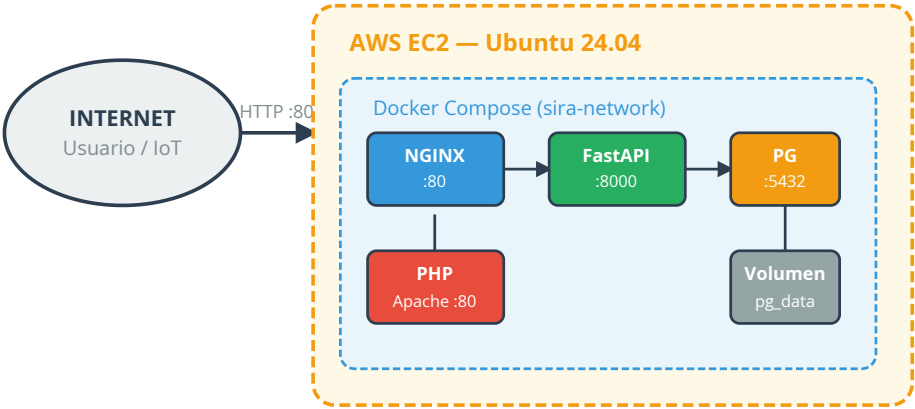
9. Despliegue en Producción (AWS)

9.1 Infraestructura Cloud

El sistema SIRA ha sido desplegado con éxito en **Amazon Web Services (AWS)** para la demo de la defensa del TFG.

Parámetro	Valor
Servicio	Amazon EC2
Tipo de instancia	t2.micro (capa gratuita)
Sistema Operativo	Ubuntu Server 24.04 LTS
RAM	1 GB (+ 2 GB Swap configurado)
Acceso	SSH (puerto 22) + HTTP (puerto 80)

9.2 Diagrama de Despliegue Cloud



9.3 Proceso de Despliegue

```
# 1. Configurar swap (instancia t2.micro con 1 GB RAM)
sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile

# 2. Instalar Docker y Docker Compose
sudo apt update && sudo apt install -y docker.io docker-compose

# 3. Clonar el repositorio y configurar el entorno
git clone https://github.com/JuanRisueno/SIRA_Project.git
cd SIRA_Project && cp .env.example .env && nano .env

# 4. Levantar todos los servicios
docker-compose up -d --build
```

Acceso: `http://<IP_PUBLICA_AWS>:80`

10. Análisis de Riesgos y Mitigaciones

#	Riesgo	Categoría	Impacto	Medida Implementada
1	Fallo del servidor durante la demo	Infraestructura	Alto	Script de arranque automático (<code>docker-compose up -d</code>)
2	Desajuste de zona horaria en gráficas	Base de Datos	Medio	<code>TZ=Europe/Madrid</code> en todos los contenedores Docker
3	Saturación del servidor por simulador	Rendimiento	Medio	Intervalo mínimo de 5-10 segundos entre inserciones
4	Pérdida de datos del formulario por meta-refresh	UX	Bajo	Auto-refresh solo activo en páginas de monitorización
5	Introducción de valores imposibles por el usuario	Validación	Medio	Validación doble: Pydantic (backend) + PHP (frontend)
6	Ataque de inyección SQL	Seguridad	Crítico	ORM SQLAlchemy parametrizado (0 SQL concatenado)
7	Robo de token JWT	Seguridad	Alto	Token en sesión PHP, no en localStorage
8	Instancia EC2 sin memoria suficiente	Cloud	Alto	2 GB Swap + monitorización de memoria
9	Corrupción o pérdida de datos	Base de Datos	Crítico	Sistema de backups graduales (Anual/Mensual/Diario) con rotación

11. Planificación y Línea Temporal

El proyecto se ha desarrollado en **6 fases** a lo largo del curso 2025-2026:

Fase	Nombre	Estado	Responsables Principales
I	Infraestructura y Base de Datos	COMPLETADA	Juan, Jorge, Alfonso
II	Desarrollo del Backend (FastAPI)	COMPLETADA	Juan
III	Interfaz Web y Autenticación	COMPLETADA	Jorge, Juan, Alfonso
IV	Simulación de Sensores y Cultivos	COMPLETADA	Juan, Jorge, Alfonso
V	Seguridad Avanzada (Iron Fortress)	COMPLETADA	Juan, Jorge
VI	Despliegue en AWS y Cierre	COMPLETADA	Juan, Jorge

Hitos Superados:

- Punto de Control 1: Servidor y BD funcionando correctamente.
- Punto de Control 2: Interfaz web funcional y conexión segura con la API.
- Punto de Control 3: Sistema de control y simulación validado.
- Punto de Control 4: Sistema protegido y listo para la defensa.
- Punto de Control 5: Proyecto desplegado y funcionando en la nube.

12. Auditoría Final y Conclusiones

12.1 Resultado de la Auditoría Técnica

Tras la revisión final de todos los componentes del sistema:

Área	Estado	Observaciones
Documentación	Completa	Guías de infraestructura, seguridad, BD y defensa redactadas.
Backend (FastAPI)	Estable	Organización modular, simulación IoT validada, JWT operativo.
Frontend (PHP/CSS)	Funcional	Glassmorphism, diseño oscuro, SSR sin dependencias JS externas.
Base de Datos (PostgreSQL)	Normalizada	Esquema 3FN, borrado lógico, backups configurados.
Infraestructura (Docker/Nginx)	Productiva	Despliegue local y AWS verificado.
Seguridad (Iron Fortress)	Auditada	Bcrypt, JWT, SID, Sliding Window, historial de contraseñas.
Despliegue AWS	Operativo	EC2 t2.micro con Swap, Security Groups configurados.

12.2 Conclusión

El sistema SIRA representa una solución tecnológica completa y funcional para la gestión automatizada del riego en explotaciones agrícolas. El proyecto demuestra la integración de competencias clave del ciclo ASIR:

- **Redes y Servidores:** Nginx como proxy inverso, Docker, AWS EC2.
- **Programación:** API REST con FastAPI (Python) y renderizado SSR con PHP.
- **Bases de Datos:** Diseño relacional normalizado en PostgreSQL.
- **Seguridad:** Autenticación JWT, hashing Bcrypt, control de sesiones.
- **Despliegue en Nube:** Instancia EC2, Security Groups, configuración de producción.

Estado del Proyecto: **TERMINADO — Versión 1.0 Final (Mayo 2026)**

13. Autoría y Roles del Proyecto

El desarrollo integral de SIRA ha sido liderado por el siguiente equipo técnico:

Integrante	Rol	Áreas de Responsabilidad
Juan Risueño	Backend Lead / DevOps	Arquitectura de sistemas, API FastAPI, protocolo Iron Fortress, despliegue AWS, Docker/Nginx
Jorge Pedro López	Coordinador / Frontend	Coordinación del proyecto, diseño E/R, desarrollo PHP, hardware IoT, efectos visuales CSS
Alfonso Navarro	BD / CSS / Docs	Esquema SQL y normalización, scripts de datos, arquitectura CSS, documentación técnica del TFG

El proyecto SIRA es el resultado de más de 6 meses de trabajo colaborativo y aprendizaje. Cada decisión técnica documentada en este dossier ha sido tomada conscientemente, con base en los principios aprendidos durante el ciclo formativo de ASIR.

SIRA Project — Dossier Técnico Maestro

Versión 1.0 Final — Mayo 2026

Trabajo de Fin de Grado — ASIR